WSC-LLM: Efficient LLM Service and Architecture Co-exploration for Wafer-scale Chips

Zheng Xu
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
xuz22@mails.tsinghua.edu.cn

Dehao Kong
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
kdh24@mails.tsinghua.edu.cn

Jiaxin Liu
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
jiaxin-l24@mails.tsinghua.edu.cn

Jinxi Li
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
ljx23@mails.tsinghua.edu.cn

Jingxiang Hou
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
houjx22@mails.tsinghua.edu.cn

Xu Dai
Shanghai AI Laboratory
Shanghai, China
daixu@pjlab.org.cn

Chao Li
Shanghai Jiao Tong University
Shanghai, China
lichao@cs.sjtu.edu.cn

Shaojun Wei
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
wsj@tsinghua.edu.cn

Yang Hu*
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
hu_yang@tsinghua.edu.cn

Shouyi Yin
Tsinghua University
School of Integrated Circuits, BNRist
Beijing, China
Shanghai AI Laboratory
Shanghai, China
yinsy@tsinghua.edu.cn

Abstract

The deployment of large language models (LLMs) imposes significant demands on computing, memory, and communication resources. Wafer-scale technology enables the high-density integration of multiple single-die chips with high-speed Die-to-Die (D2D) interconnections, presenting a promising solution to meet these demands arising from LLMs. However, given the limited wafer area, a trade-off needs to be made among computing, storage, and communication resources. Maximizing the benefits and minimizing the drawbacks of wafer-scale technology is crucial for enhancing the performance of LLM service systems, which poses challenges to both architecture and scheduling. Unfortunately, existing methods cannot effectively address these challenges.

To bridge the gap, we propose WSC-LLM, an architecture and scheduling co-exploration framework. We first define a highly configurable general hardware template designed to explore optimal architectural parameters for wafer-scale chips. Based on it, we capitalize on the high D2D bandwidth and fine-grained operation advantages inherent to wafer-scale chips to investigate optimal disaggregated scheduling strategies, effectively addressing the highly dynamic demands of LLM workloads. Compared to the state-of-the-art (SOTA) LLM service systems, WSC-LLM can achieve an average overall performance improvement of 3.12× across various LLM models and datasets. Moreover, we leverage WSC-LLM to reveal intriguing insights about wafer-scale architecture design and the execution of LLM workloads.

CCS Concepts

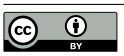
• **Computer systems organization** → **Architectures**; • **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Machine learning*.

Keywords

scheduling, architecture, wafer-scale chips, large language model

ACM Reference Format:
Zheng Xu, Dehao Kong, Jiaxin Liu, Jinxi Li, Jingxiang Hou, Xu Dai, Chao Li, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. WSC-LLM: Efficient LLM Service and Architecture Co-exploration for Wafer-scale Chips. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3695053.3731101

*Corresponding author.



ISCA '25, Tokyo, Japan

ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731101

1 Introduction

In recent years, large language models (LLMs) based on the transformer architecture have emerged as a driving force behind advancements in artificial intelligence (AI). It has demonstrated remarkable capabilities and exceptional performance across various domains, including natural language understanding and content generation [8, 9, 13, 18, 76, 77]. To achieve superior performance, the number of parameters in LLMs has increased dramatically, resulting in a substantial rise in computational resource demands. In addition to the large number of model parameters, the LLM inference process also generates a significant amount of key-value (KV) cache, further exacerbating memory requirements.

Affected by the end of Moore's Law [55] and the constraints imposed by limited photomask size [5], it is difficult for a single hardware device, such as an individual GPU or NPU, to achieve sufficient transistor scale and memory capacity to support LLMs tasks. As a result, distributed computing architectures is essential for efficient LLM inference. However, deploying LLMs across multiple devices introduces additional challenges in communication. Disaggregate LLM inference necessitates large-scale transmission of model weights and KV cache across devices, imposing demands on high-bandwidth interconnects. Currently, interconnect communication bandwidth has become the primary bottleneck limiting the performance of these LLM serving systems [10, 45, 85].

Wafer-scale chip design method [33], using advanced packaging technologies such as CoWoS [29] to expand the chip area, provides a promising solution to meet the computation, memory and communication resource demands of LLM inference and enable high-density 3D integration. Vertical integration of modules such as power supply, cooling, and I/O interfaces achieves exceptional integration density, which in turn maximizes compute density and interconnect bandwidth. Currently, wafer-scale GPU [61], UCLA & UIUC's work [60], Cerebras WSE2 [50], and Tesla's Dojo [73] have been introduced. One widely used, cost-effective, and high-yield wafer-scale approach involves integrating NPU and DRAM chiplets together [60, 61, 73]. Compared with the most advanced GPUs [7] and high-speed rack-scale fabrics (i.e., NVLinks [6]), these wafer-scale chip works can provide around 50× the number of transistors and 6× the inter-chip bandwidth.

However, this technology introduces new challenges for architectural design and LLM inference scheduling for wafer-scale chips, which are outlined below:

For architecture design, the key challenge is determining the optimal DRAM capacity per die. The weights of LLMs can reach hundreds of gigabytes [2–4, 9, 13, 18, 66, 75, 96], and KV cache storage demands are similarly substantial. As illustrated in Fig. 1(a), a higher DRAM capacity facilitates the simultaneous execution of a greater number of requests, thereby enhancing the overall quality of LLM services. However, larger DRAM capacity implies that DRAM chiplets will occupy more wafer area and demand additional Die-to-Die (D2D) interconnect interfaces, which will reduce the available compute and communication resources. All the above adverse effects are collectively referred to as *DRAM Costs*. Consequently, a trade-off arises between integrating larger DRAM capacities within a single die to support extensive weights and KV cache storage demands and smaller DRAM capacities to reduce *DRAM Costs*. Balancing this trade-off remains an unsolved challenge.

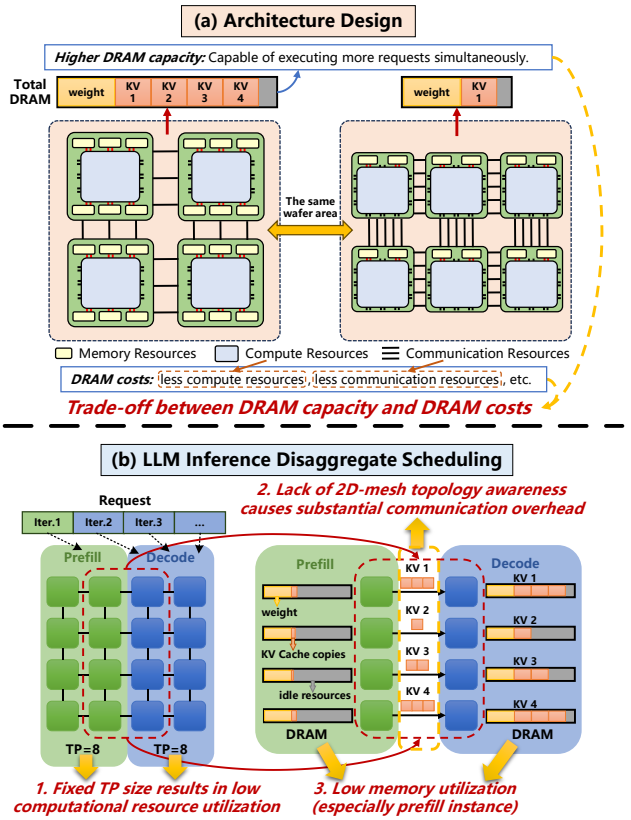


Figure 1: The challenges when LLMs meet wafer-scale chip.

For LLM inference scheduling, the main challenge lies in managing the highly dynamic nature of LLM workloads, which stems from the division of LLM request processing into two distinct phases: prefill and decoding. Resource demands of the same request in different phases vary significantly [28, 30, 62, 71, 84, 85, 100]. Consequently, using identical devices and execution strategies for both phases often leads to inefficiency. A potential solution is *disaggregated scheduling*, which partitions devices into multiple groups, each deploying an LLM instance for a specific phase and applying a phase-specific execution strategy tailored to its characteristics [30, 31, 62, 84, 100]. However, as illustrated in Fig. 1(b), it faces several limitations. First, optimizing tensor parallelism (TP) strategy and group configuration is crucial for maximizing computational performance during both the prefill and decoding phases. Nonetheless, most existing works select fixed configurations based on engineering experience, lacking precise problem definition and exhaustive exploration. Second, these solutions [30, 31, 62, 100] require KV cache transfers during the phase transition, and the lack of design for wafer-scale 2D-mesh topology may incur substantial communication overhead that cannot be masked effectively by pipeline execution. Third, due to isolation among groups, the memory of different device groups cannot be collectively utilized to store the KV cache for a single request. And most KV cache storage is concentrated in the decoding device group, leading to inefficient memory utilization.

The above analysis reveals that employing wafer-scale chips to efficiently serve LLMs introduces intricate trade-offs and challenges. This not only poses challenges for architectural design but

also necessitates efficient parallel execution strategies and scheduling solutions to fully leverage the computation, memory, and communication resources of wafer-scale chips. To address these challenges, we have made the following contributions:

- Based on a highly configurable and universal hardware template, we propose WSC-LLM, an efficient scheduling and architecture co-exploration framework for wafer-scale chips. To the best of our knowledge, this is **the first** work to achieve it.
- We specifically develop strategies tailored to the characteristics of the wafer-scale architecture to optimize tensor parallelism, resource allocation and placement, and memory management. These algorithms aim to explore the most efficient disaggregated scheduling strategies while efficiently managing KV cache storage, thereby maximizing the utilization of hardware resources on wafer-scale chips.
- We utilize WSC-LLM to reveal several intriguing insights about wafer-scale architecture design and the execution of LLM workloads as follows. (1) With the support of WSC-LLM, wafer-scale chips equipped with moderate DRAM capacity per die can effectively balance computation, storage, and communication resources, thereby delivering the best LLM service quality. (2) Supported by our advanced memory management strategies, achieving efficient LLM service requires not only high DRAM bandwidth but also sufficient communication bandwidth. A balance between memory and communication resources is essential for optimal performance. (3) Our investigation into the impact of various optimization strategies on LLM inference performance reveals that the memory optimization strategies in WSC-LLM contribute more significantly to enhancing LLM service quality compared to computation optimization strategies.
- Compared to the SOTA LLM service systems, WSC-LLM can achieve an average overall performance improvement of 3.12× across various LLM models and datasets.

2 Background

2.1 LLM Inference

Most popular large language models (LLMs) [2, 8, 9, 76, 77] employ a decoder-only Transformer architecture [78]. The model consists of a stack of transformer layers, each containing an attention layer and a feed-forward network (FFN) layer. Attention layers make tokens in a request interact with other tokens, while FFN layers process requests in a token-wise manner.

As shown in Fig. 2, LLM inference process comprises two distinct phases: prefill and decoding. During the prefill phase, the LLM model processes all input tokens in parallel upon receiving a user request, generating the first token while caching the computed key-value (KV) pairs across all transformer layers. This cached data, formally termed KV cache [63], eliminates redundant computation of historical token matrices during subsequent decoding phase. The prefill process exhibits characteristics similar to the training forward pass and is computation-intensive. In the decoding phase, it uses the last generated token and the KV cache as inputs to generate tokens iteratively until reaching the termination token. The KV cache exhibits linear growth proportional to both the model parameters and the input sequence length. For GPT-175B [13], a single request with an input sequence length of 1,024 tokens generates approximately 4.5 GB of KV cache. Consequently, the decoding phase involves massive KV cache accesses, imposing significant demands on both memory capacity and bandwidth. Furthermore, transferring KV cache data between different hardware devices imposes substantial communication bandwidth demands.

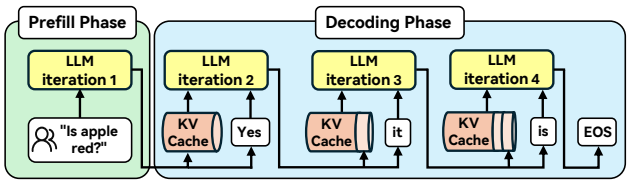


Figure 2: Prefill and decoding phase in the LLM inference.

2.2 LLM Serving System

With the increasing size of models, model parallelism has become essential for both training and inference. Model parallelism allows a model to be distributed across multiple devices. The two primary types of model parallelism used in inference are tensor parallelism (TP) [48, 54, 64, 70, 87, 88, 92, 93, 97] and pipeline parallelism (PP) [12, 21, 31, 36, 49, 56]. TP divides tensors along specific dimensions, distributing the partitioned computations across multiple devices to allow concurrent execution of different parts of an operator. In contrast, PP distributes different operators in the computational graph across devices, representing an orthogonal approach. Typically, TP is more effective than PP for devices within the same large high-bandwidth domain [62]. Given the wafer-scale chip's capability to deliver a large high-bandwidth domain across the entire wafer, our optimization efforts focus primarily on TP.

In real-time online LLM service systems [10, 45, 85, 91], continuous batching is a commonly used technique that processes the prefill and decoding phases of multiple requests in each iteration. However, since the prefill and decoding phases are constrained by computation and memory bandwidth, respectively, continuous batching results in prefill-decoding interference. To address this issue, disaggregated inference [31, 62, 72, 100] separates prefill and decoding phases onto different devices. While this approach can enhance service quality and achieve simultaneous optimization of the prefill and decoding phases, challenges such as imbalanced resource allocation, additional communication overhead, and suboptimal memory utilization persist.

2.3 Wafer-scale Chip

Benefiting from advances in packaging technologies like Through-Silicon Vias (TSV), Chip on Wafer on Substrate (CoWoS), Redistribution Layer (RDL) [26, 29, 32, 34, 46, 47, 95], wafer-scale chip is a feasible and promising architectural solution to further scaling up chip sizes. There are two mainstream technical routes. The first is monolithic integration, where all compute dies and interconnects are directly integrated on wafer-scale chip [50, 60, 68]. The other method is chiplet-based integration, in which compute dies are fabricated separately and then integrated into the wafer-scale chip along with interconnects [60, 61, 69, 81]. This approach enables flexibility, configurability, and high scalability through integration

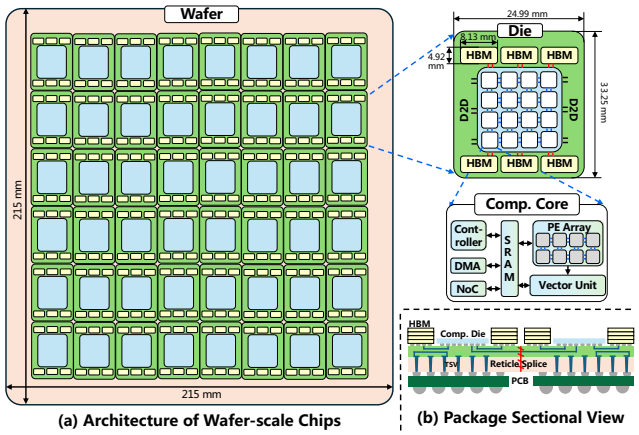


Figure 3: Architecture of wafer-scale chips.

with diverse devices on the wafer. Moreover, compared to the first approach, chiplet-based wafer-scale chips made through mature technology and production enhance yield and reduce redundancy. Therefore, we adopt the chiplet-based integration.

(1) Overall Architecture: As depicted in Fig. 3(a), the hardware template architecture consists of three hierarchical levels: wafer, die, and computing core. The architecture employs a mesh topology for the 215 mm×215 mm wafer, utilizing Die-to-Die (D2D) interconnect interfaces to facilitate communication between adjacent dies. This configuration supports various communication types, such as Die-to-Die, Die-to-DRAM, and DRAM-to-Die. Although alternative topologies, such as torus and fully-connected designs, are feasible, the constraints on interconnect distances and the need to maintain signal integrity often reduce bandwidth and efficiency. Therefore, the mesh topology remains the optimal choice.

(2) Die Architecture: Each die, measuring 24.99 mm×33.25 mm, integrates both computational and storage functionalities. This integration marks a significant shift from conventional chiplet designs [15, 69, 74], which typically separate these functions. As shown in Fig. 3(a), each die comprises multiple computing cores, on-die memory (DRAM chiplets), NoC interconnections, and D2D interconnect interfaces. The NoC interconnections also follow a mesh topology, facilitating high-speed communication between cores. The placement of D2D interfaces around the die periphery contributes to a scalable architecture.

(3) Computing Core Architecture: As illustrated in Fig. 3(a), each computing core is responsible for executing computational tasks. The controller manages core operations using pre-defined instructions and coordinates data transfers across external cores and DRAM. DMA and NoC systems handle communication between cores, ensuring efficient data flow within the framework. Each core has globally shared SRAM, while the PE array and vector unit performs General Matrix Multiplication (GEMM)/General Matrix-Vector Multiplication (GEMV) and vector/scalar operations, respectively.

(4) Configurable Parameters: Our hardware template offers extensive configurability and scalability suited to wafer-scale chips, including the number of dies in both horizontal and vertical directions, DRAM and SRAM bandwidth and capacity, as well as NoC and D2D bandwidth. Additionally, the parameters of the compute die are also configurable, encompassing the number of cores in the horizontal direction, the number of cores in the vertical direction, and the number of MACs in the PE array. The design of the PE array and its dataflows are derived from existing works [14, 17, 19, 27, 41, 43, 74, 80, 90].

(5) Package Sectional View: As illustrated in Fig. 3(b), the wafer-scale chip employs an interposer layer to enable high-speed communication both between compute dies and between compute dies and DRAM. This design avoids low-speed communication by bypassing the multiple dielectric transitions typically encountered from the chip, through the interposer and substrate, to the PCB. Consequently, this configuration improves communication speed (higher D2D bandwith), reduces transmission distances, and enhances signal integrity.

3 Motivation

3.1 Trade-off Introduced by DRAM capacity

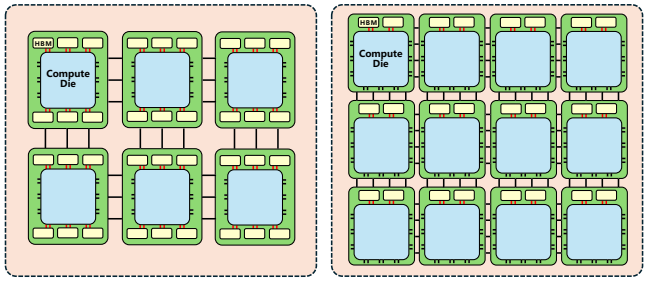
LLM workloads impose substantial demands on computation, storage, and communication resources. Although wafer-scale technology can effectively scale monolithic devices, it remains constrained by the limits of the wafer size. For example, the maximum usable area of a 12-inch wafer is approximately 46225 mm^2 (215 $mm \times 215$ mm) [33]. Consequently, wafer-scale chips necessitate trade-offs among various hardware resources. However, existing research has not examined this. In this section, we discuss the benefits and drawbacks of increasing DRAM capacity on wafer-scale chips.

Fig. 4(a) and (b) illustrate large and small DRAM capacities configurations within a single die, respectively. In Fig. 4(a), DRAM chiplets are positioned on both sides of the compute die, while in Fig. 4(b), they are arranged on only one side. Increasing the number of DRAM chiplets allows for higher DRAM bandwidth by utilizing more compute-die interconnect interfaces.

As shown in Fig. 4, expanding DRAM capacity on wafer-scale chips offers two primary advantages. First, increased DRAM capacity allows for storing more weights and KV caches, theoretically enabling the simultaneous processing of a higher number of LLM user requests. Second, it can enhance DRAM bandwidth, which is particularly crucial for decoding, as this phase is memory-bound. Given that the decoding phase often constitutes the majority of the runtime across most workloads, higher DRAM bandwidth can significantly improve the quality of service for LLMs by reducing memory-related bottlenecks.

However, Fig. 4 also highlights its associated disadvantages. First, the number of interconnect interfaces is fixed for the same computing die. An increase in the number of DRAM chiplets results in a higher proportion of computing-die interfaces being occupied by memory access bandwidth, reducing the number of interfaces available for Die-to-Die (D2D) communication. Consequently, the D2D communication bandwidth decreases, thereby limiting inter-die data transfer efficiency. Second, additional DRAM chiplets occupy more wafer area, leading to a reduction in the total number of dies per wafer and consequently lowering the overall computational capacity of the system.

These trade-offs underscore the critical balance between DRAM bandwidth and capacity, D2D bandwidth, and computational resources for achieving optimal LLM performance.



Trade-offs Introduced by DRAM capacity on wafer-scale chips			
Advantages		Disadvantages	
Larger storage space	Higher DRAM bandwidth	Fewer dies on wafer	Lower D2D bandwidth

Figure 4: Trade-offs Introduced by DRAM capacity on wafer-scale chips.

3.2 Problems with existing LLM serving systems

Although existing LLM serving systems [30, 31, 62, 100] recognize the differences between the prefill and decoding phases and map them to separate machines, they still face the following issues.

First, these systems overlook the exploration of optimal TP granularity and resource allocation and placement for both prefill and decoding instances. Current approaches typically set the TP size to a fixed value such as 8 GPUs (one node) and rely on engineering experience for instance allocation. Fig. 5(a) shows the average iteration time for the prefill and decoding phase in LLaMA3-70B under the optimal TP strategy across different TP sizes and batch sizes. Detailed TP strategies and compute die configurations are provided in Section 4.5.1 and Section 5.1. The result reveals that increasing TP size accelerates the computation-intensive prefill phase while saving DRAM space. However, in the decoding phase with lighter computational demands, larger TP size may reduce performance due to additional communication overhead. Thus, the optimal TP size may differ between two phases, necessitating distinct execution strategies. Furthermore, the optimal allocation and placement of hardware resources between two phases remains an open problem requiring systematic exploration.

Second, these systems require KV cache transfers from prefill to decoding instances, resulting in substantial inter-node communication overhead. Due to the limited communication bandwidth across GPU clusters, this overhead cannot be effectively overlapped with computation. While wafer-scale chips provide higher bandwidth that holds promise for solving this probelm, their 2D-mesh topology differs fundamentally from the all-to-all topology of GPU clusters. Without topology-aware scheduling, communication traffic becomes imbalanced, leading to severe congestion on certain links and making high communication overhead perisist.

Third, the memory utilization of these systems is suboptimal. Fig. 5(b) illustrates peak DRAM utilization for prefill and decoding instances when running the LLaMA3-70B model and the conversation dataset on wafer-scale chips, where each die is equipped with 96GB of DRAM. Details of the die configurations and workloads are provided in Section 5.1. DRAM utilization in prefill instances is significantly low due to the KV cache transfer. Furthermore, decoding instances also cannot utilize all DRAM resources because they cannot store the KV cache of the same request across instances.

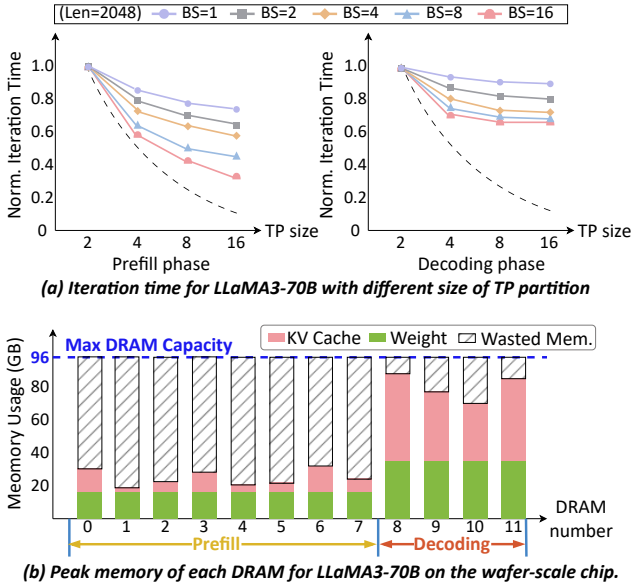


Figure 5: Request performance under different TP sizes in different phases and memory utilization in prefill and decoding instances.

4 WSC-LLM Framework

4.1 WSC-LLM Overview

WSC-LLM is a scheduling and architecture co-exploration framework for wafer-scale chips, marking the first work of its kind. As depicted in the left of Fig. 6, the inputs for WSC-LLM consist of architecture parameter candidates, LLM models, and workload. Within the constraints of wafer area limitations, architecture parameter candidates are derived from arbitrary combinations of configurable parameters (introduced in Section 2.3).

All architectural candidates are exhaustively explored by the Scheduling Engine to achieve optimal LLM service quality on wafer-scale chip with varying architecture parameters. At the core of the Scheduling Engine is the Central Scheduler, which leverages the TP Engine to explore the optimal TP configurations, resource allocation and placement strategies while managing the allocation of user requests within the Execution Engine. The exploration of the optimal TP configuration and resource allocation (Section 4.2.1), as well as the resource placement strategy (Section 4.2.2), are both executed offline. Therefore, they do not introduce any additional system overhead during the online execution of the Central Scheduler. Furthermore, the Memory Scheduler determines the optimal KV cache storage location, ensuring efficient utilization of storage resources and minimizing additional communication overhead.

4.2 Central Scheduler

4.2.1 Optimal Resource Partition Strategy. Motivated by the need to support the dynamic nature of LLM workloads efficiently, we map the prefill and decoding phases to separate hardware resources, creating distinct instances of prefill and decoding phases. However, this approach introduces two new unresolved issues: determining the optimal instance size and TP setup for each phase and selecting

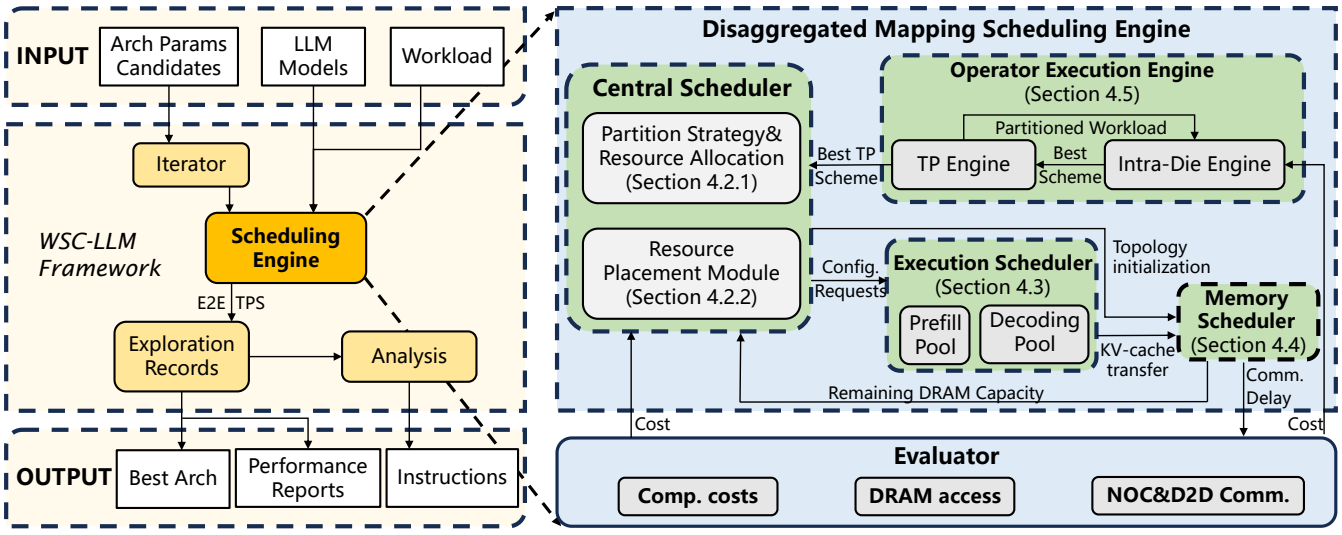


Figure 6: WSC-LLM framework.

the appropriate number of instances for the prefill and decoding phases.

To tackle these challenges, we propose Algorithm 1, which systematically optimizes the resource partitioning for prefill and decoding phases. The algorithm first optimizes the prefill and decoding instance size along with their TP setup independently to achieve phase-level optimal per-die throughput. Subsequently, to achieve globally optimal performance, we employ a replication strategy, which means the instance size for each phase is uniform, and the same TP strategy is applied across the same phase. At last, the number of prefill and decoding instances is determined based on the traffic balance between the prefill phase and the decoding phase.

Given the model M, constraints on the number of dies per instance D, memory capacity of a die c, workload W, TP partition strategy S (Detailed in Section 4.5.1), and the total number of dies N, the algorithm can determine the best partitioning setup $prefill\_setup$ and $decoding\_setup$) and allocates the appropriate number of instances for each phase (NP and ND). The workload W is obtained by resampling the distribution after fitting the test dataset (introduced in Section 5.1).

The algorithm iterates through all feasible instance sizes, which are constrained by the die limit per instance (line 3) and the total memory capacity (line 4). Additionally, it is important to note that, in order to satisfy the communication requirements imposed by the 2D-mesh topology of wafer-scale chips, the arrangement of dies within each instance must be a rectangular shape, which means that linear or other irregular shapes are not permitted. The algorithm then enumerates all possible TP partition strategies for a fixed instance size (line 5). Subsequently, we determine the goodput for the prefill phase by the prefill workload $W\_prefill$ (line 7), and similarly, determine it for the decoding phase by the decoding workload $W\_decoding$ (line 8). Notably, the *test* function operates as a simulator that executes workload on a single instance, without incorporating cross-instance scheduling. While throughput is influenced by both placement and scheduling strategies, the goodput measured here solely reflects the performance within a single instance. It represents the upper performance bound that can be achieved after optimized global scheduling, helping identify setup with greater potential for scheduling improvements. Once the optimal setup for both phases are identified (lines 9-13), we replicate them to achieve the optimal overall performance by balancing the throughput of each phase. We determine the number of prefill and decoding instances according to the total number of dies N and the proportion of their throughput (lines 18-19).

The complexity of Algorithm 1 is $O(DS)$, where D is the die limit per instance, and S is the number of pre-defined TP partition strategies. The search space remains manageable, with a solving time of only a few minutes, even in the largest instance and LLM model scenario.

4.2.2 Optimal Resource Placement Strategy. In Section 4.2.1, we explored the optimal instance sizes and numbers for the prefill and decoding phases. However, due to the characteristics of the 2D-mesh topology in wafer-scale chips, it is also necessary to determine the optimal placement of each instance.

Since the results of prefill instances (i.e., KV cache) are used by decoding instances, data transfer only occurs from prefill instances to decoding instances. Given this scenario, the objective is to minimize the total distance between each prefill instance and its nearest decoding instance while avoiding overlapping transmission paths. To achieve this, we adopt a strategy that prioritizes central placement for decoding instances: the decoding instances are placed in the central region first, followed by the placement of prefill instances around the perimeter. If placement schemes are not unique, we select the one that minimizes the total communication cost. This selection is formalized as:

$$\text{TransferCost} = \sum_{i=0}^{NP-1} \min_{j=0}^{ND-1} \text{Distance}(P_i, D_j) \tag{1}$$

where NP and ND denote the number of prefill and decoding instances, respectively. P_i and D_j represent the center positions of prefill and decoding instances, where $i \in [0, NP-1]$ and $j \in$

Algorithm 1: Optimal Resource Partition Algorithm

Input: Model M, dies limit D, memory capacity per die c, workload W, TP split strategy S, total dies N.
Output: $prefill\_setup$, $decoding\_setup$, number of prefill instances NP, number of decoding instances ND.

$prefill\_setup \leftarrow \emptyset$
$decoding\_setup \leftarrow \emptyset$
for $ins \in \{1, 2, \ldots, D\}$ **do**
  if $\frac{M.weight}{ins} < N \cdot c$ **then**
    for $s \in S$ **do**
      $TP \leftarrow (M, ins, s)$
      $prefill\_goodput \leftarrow test(TP, W\_perfill)$
      $decoding\_goodput \leftarrow test(TP, W\_decoding)$
      if $\frac{prefill\_setup.goodput}{prefill\_setup.TP.ins} < \frac{prefill\_goodput}{ins}$ **then**
        $prefill\_setup \leftarrow (TP, prefill\_goodput)$
      end
      if $\frac{decoding\_setup.goodput}{decoding\_setup.TP.ins} < \frac{decoding\_goodput}{ins}$ **then**
        $decoding\_setup \leftarrow (TP, decoding\_goodput)$
      end
    end
  end
end
$NP \leftarrow \left\lceil \frac{N \times prefill\_setup.goodput}{prefill\_setup.goodput + decoding\_setup.goodput} \right\rceil$
$ND \leftarrow N - NP$
return $prefill\_setup$, $decoding\_setup$, NP, ND

$[0, ND-1]$. *Distance* refers to the shortest path between prefill and decoding instances. In cases where multiple shortest paths exist, all such paths are enumerated. Among these, a non-overlapping path configuration is selected to achieve optimal matching.

It is noteworthy to add a hyperparameter $\alpha \geq 1$ when there is the reuse of communication links. For example, in Fig. 7(a), the Distance$(P_1, D_1) = 2$ hops, whereas Distance$(P_5, D_1) = (2\alpha + 2)$ hops, because the paths from P_5 to D_1 and P_1 to D_1 share 2 hops. The value of α depends on the number of shared links and the ratio between D2D and DRAM bandwidths. For instance, in the scenario depicted in Fig. 7(a), the number of shared links is 2 (for P_1 and P_5 to D_1). Therefore, if the D2D bandwidth exceeds twice the DRAM bandwidth, then $\alpha = 1$.

Fig. 7(a) and (b) illustrate the placement results of applying the random strategy and our decoding-centered strategy, respectively, on a wafer-scale chip with 48 dies, where NP is 8 and ND is 4. The corresponding TransferCost are $(8\alpha + 16)$ hops and 16 hops, respectively, demonstrating the superiority of our approach.

4.3 Execution Scheduler

4.3.1 Prefill Pool. In the prefill pool, each prefill instance maintains a queue QP_i to manage user requests assigned to it. Each queue employs a first-come-first-serve (FCFS) strategy for scheduling batched requests. To maximize hardware utilization when handling irregular prefill prompt lengths, we adopt the chunked prefill technique [10]. This approach divides long prompts into uniformly sized chunks, which are processed iteratively during the prefill phase. By combining batching with chunked prefill technology, we can efficiently handle input prompts of varying lengths across diverse hardware resources.

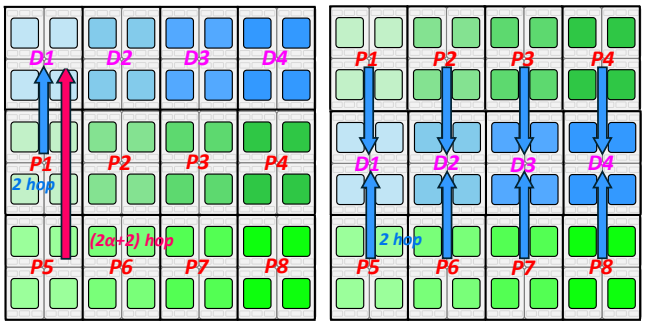


Figure 7: Different resource placement strategies.

Taking mapping scheduling in Fig. 8 as an example, the prefill pool contains three queues with 8 dies. QP_1 handles requests 1 and 2, QP_2 manages request 3 and newly arrived request 6, and QP_3 processes requests 4 and 5. The central scheduler has allocated request 6 to the least occupied queue, which is QP_2. Similarly, requests 7 and 8 are allocated to QP_3 and QP_1, respectively. Due to the long input tokens of requests 3 and 4, the execution of each request individually can reach the hardware's computation intensive region. Consequently, the batching strategy is not employed for these requests, thereby avoiding the additional time required to batch with subsequent requests. Specifically, QP_2 will only process request 3 and not request 6, while QP_3 will exclusively execute request 4 without handling request 5.

4.3.2 Decoding Pool. In the decoding pool, each decoding instance maintains a queue QD_i to manage user requests. Each queue QD_i also utilizes the FCFS method to process batched requests. Given the lower computation requirements of the decoding phase, we aim to maximize the efficiency by batching tokens whenever possible. To achieve this, we utilize the continuous batching technique [91], which allows parallel computation of batched tokens for non-attention operations while sequentially computing attention operations.

As shown in Fig. 8, we assume the decoding pool has three queues and each queue is equipped with 4 dies. QD_1 holds requests 1 and 2, QD_2 includes requests 3, and QD_3 contains requests 4 and 5. When generating the next token, requests 1 and 2 are inserted after request 8. These three requests are batched together and executed on DD dies. QD_2 executes requests 6 and 3. Since request 5 only decodes once and terminates, QD_3 runs requests 7 and 4.

4.4 Memory Scheduler

Wafer-scale chips offer high D2D bandwidth, typically exceeding DRAM access bandwidth. Thus, in the absence of D2D link congestion, cross-die DRAM read and write operations are constrained only by DRAM bandwidth rather than D2D bandwidth. Leveraging this advantage of the wafer-scale architecture, the extensive inter-die transfer of KV cache can be overlapped by its DRAM access,

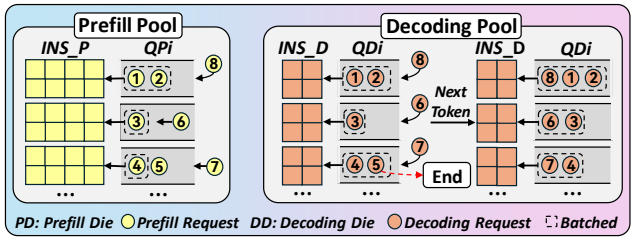


Figure 8: The architecture of the Execution Scheduler.

allowing us to fully utilize all DRAMs on the wafer for KV cache storage to achieve higher performance of LLM service.

As illustrated in Algorithm 2, we implement an optimal KV cache placement algorithm to minimize the additional D2D transmission overhead by allocating the optimal storage location for each request's KV cache. Given a group requests R, these are executed by prefill instance P_i and decoding instance D_i. The DRAM set is represented by D. The algorithm aims to find the optimal storage location of KV cache for each request.

The algorithm first identifies the associated set of DRAMs D' that require no additional communication, which is determined by the shortest transmission paths of prefill and decoding instances and their respective locations (line 1). For instance, in Fig. 7(b), the associated DRAM set for P_1 and D_1 includes all DRAMs within D_1, P_1 and P_2. The algorithm then sorts these DRAMs based on their locations and available capacities, forming a priority queue Q where location priority supersedes residual capacity (line 2).

Subsequently, for each request (line 3), an empty storage set S_i is initialized (line 4), and the total remaining DRAM capacity of Q is verified to ensure it meets the storage requirement of KV cache (line 5). If the total capacity of all remaining DRAMs in Q is insufficient to satisfy the KV cache demands of r_i, the memory scheduler notifies the central scheduler to stop the execution of r_i and subsequent requests on the prefill instance P_i (line 6). Otherwise, the algorithm iteratively allocates DRAM resources to the request r_i (line 8-18). The capacity of each DRAM unit is reduced by the portion utilized during the allocation process. After calculating the remaining capacity, the DRAM unit is reinserted into the queue Q, where it is reordered to maintain the queue's dynamic prioritization for subsequent allocations. The allocation loop continues until the entire KV cache requirement of r_i is met.

The complexity of Algorithm 2 can be approximately regarded as $O(n)$, where n represents the number of batched requests executed in a instance, typically not exceeding 16. Therefore, the decision-making process of the memory scheduler is highly efficient, ensuring that it does not hinder the execution of WSC-LLM.

4.5 Operator Execution Engine

To efficiently execute LLM operators, computational tasks must be partitioned and mapped across different dies and cores. As illustrated in Fig. 9, taking the GEMM-type operator as an example, it can be partitioned along four dimensions: B, S, H, and K. B represents the batch size, S represents the sequence length, H represents the hidden size, and K represents the other dimension of the weight matrix apart from H. Other operator types, such as Vector (e.g., nonlinear activation functions), feed-forward network (FFN), and Attention mechanism in Transformer architectures [78], can also

Algorithm 2: Optimal KV Cache Placement Algorithm

Input: Batched requests set $R = \{r_1, r_2, \ldots, r_n\}$, DRAM set $D = \{d_1, d_2, \ldots, d_m\}$, prefill/decoding instance P_i/D_j.
Output: KV cache storage set $\{S_1, S_2, \ldots, S_n\}$.

```
1  D' ← Relevant(P_i, D_j)
2  Q ← Sort(D'.location, D'.capacity)
3  for i ∈ {1, 2, ..., n} do
4      S_i ← ∅
5      if Σ_{d'∈Q} d'.capacity < r_i.kv then
6          break
7      end
8      else
9          while r_i.kv > 0 do
10             d' ← Q.pop()
11             S_i.add(d')
12             if d'.capacity > r_i.kv then
13                 d'.capacity −= r_i.kv
14                 Q.insert(d')
15                 break
16             end
17             else
18                 r_i.kv −= d'.capacity
19             end
20         end
21     end
22 end
23 return {S_1, S_2, ..., S_n}
```

be partitioned across multiple dimensions in a manner similar to the GEMM operator.

The computational task allocation strategy is structured into two levels: (1) Die-level (Section 4.5.1): The TP Engine partitions the overall computational task into sub-computation tasks and maps them across different dies. (2) Core-level (Section 4.5.2): Each subtask is further partitioned into atom-computation tasks, with each core executing a single atom-computation per time unit.

4.5.1 TP Engine. We classify TP partition strategies according to the number of partition dimensions in sub-computation tasks, categorizing them into two main types: basic TP partition strategy and hybrid TP partition strategy.

Basic TP partition strategy refers to partitioning an operator along a single dimension. Taking the GEMM operator as an example, it involves four dimensions: B, S, H, and K, corresponding to four basic TP partition strategies, respectively. Partitioning along B, S and K requires All-gather communication among different dies within the same instance, and partitioning along H involves All-reduce communication due to the summation operations.

Hybrid TP partition strategies involve operator partition across multiple dimensions. While each additional partition dimension incurs extra All-gather or All-reduce communication overhead, it enables larger TP and instance sizes. Since each instance retains a full copy of the model weights, increasing the instance size reduces the memory footprint of model weights in DRAMs, and provides

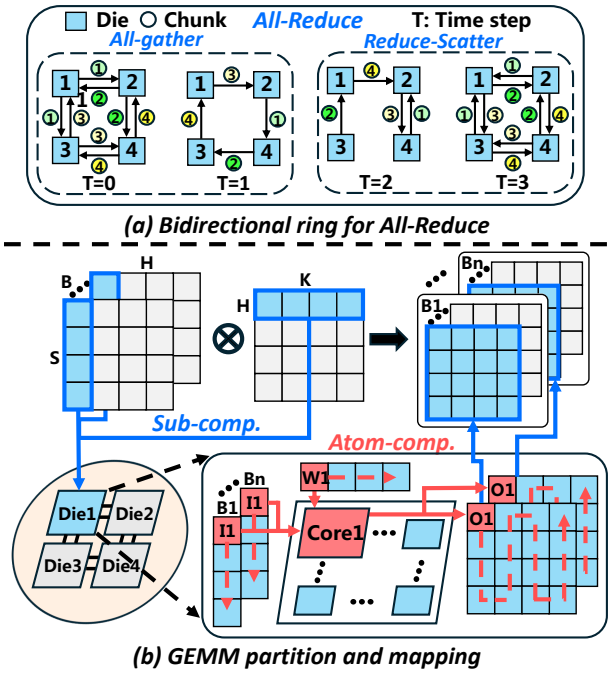


Figure 9: All-reduce communication process and the two-level operator partition and mapping.

the Memory Scheduler with greater optimization space, ultimately leading to potential improved overall system performance.

TP partition strategies for other operator types follow a similar definition as for GEMM. Notably, for the Attention operator, we prioritize partition it along the Q, K, and V heads, which form the basic TP partition strategy. LLaMA3-70B employs the Grouped Query Attention (GQA) mechanism, featuring only eight KV heads. As shown in Fig. 5, when TP size is 16, a hybrid TP partition strategy is required. In this case, each K and V head is further partitioned along an additional dimension, such as H, ensuring that each die retains only half of the K and V head.

We implement the All-gather and All-reduce communications in the TP process using the bidirectional Ring algorithm [82], which is well-suited for the 2D-mesh communication network of wafer-scale chips. Fig. 9(a) illustrates the All-reduce communication process when TP size is 4, requiring a total of four time steps. All-reduce can be decomposed into two phases: Reduce-Scatter and All-gather, with each phase comprising two time steps.

4.5.2 Intra-Die Engine. Upon receiving the sub-computation task assigned by the TP Engine, each die further partitions it along multiple dimensions, such as B, S, H, and K, to generate atom-computation tasks. Each atom-computation task must comply with the SRAM constraints of the computing core and is mapped to a single computing core for execution within one time step.

Fig. 9(b) illustrates the complete two-level task allocation process. First, the TP Engine partitions the operator along the H dimension and maps the sub-computation task to Die1. Then, further partitioning along the S and K dimensions is performed, mapping each atom-computation task based on the Weight Stationary (WS) approach, iteratively completing the computation.

It is important to note that the WS mapping strategy shown in Fig. 9(b) may not always be the most efficient, as it might overlook opportunities to reuse SRAM within computing cores to reduce system overhead. Therefore, we further explore optimal intra-die mapping strategies to achieve the best execution performance. To exploring the mapping space of atom-computation tasks efficiently, we employ heuristic algorithms [24, 79, 99] to make full use of the intra-layer pipelining [23, 27, 35, 42–44, 52, 80, 86].

4.6 Evaluator

The evaluator in the WSC-LLM framework is built upon ASTRA-sim [83], an open-source distributed training simulator developed using real hardware measurements. Our intra-die engine (Section 4.5.2) serves as the interface for global performance evaluation, analyzing the execution time of computation tasks within a single die. Based on the prior work, we introduce two significant efforts.

First, we extend ASTRA-sim to support the inference process of LLMs, which is inherently dynamic across iterations, unlike the relatively static nature of the training process. This extension enables independent simulation of each iteration in the LLM inference process and aggregates the statistical results across all iterations at the end, thereby achieving iteration-level hardware simulation.

Secondly, to meet the real-time requirements of LLM inference, we pre-build and store the mapping lookup table offline. First, we explore the optimal intra-die mapping strategy for representative configurations in test datasets, including different models, phases, token counts, and batch sizes. For each configuration, key results such as latency, DRAM access volume, and communication overhead are recorded. Subsequently, these results are used to train a DNN to model the relationship between input metrics and outcomes. Finally, the DNN is employed to estimate results across all possible configurations within the test dataset to complete the construction of the mapping lookup table. During the online execution of WSC-LLM, the lookup table remains static, requiring only read operations with negligible overhead. Existing simulator works [37, 89] validate the feasibility of this approach, demonstrating that the error of the fitted results is within a controllable range.

5 Evaluation

5.1 Experiment Setup

5.1.1 Basic Hardware Configurations. To facilitate the exploration of wafer-scale chip architecture, we configured the parameters of the compute die according to the setup described in Fig. 3. The compute die measures 21.92 mm × 22.81 mm and consists of a 16 × 16 array of Dojo-style[73] compute cores. It is manufactured using TSMC's 7 nm process technology and operates at a frequency of 1 GHz. The interconnect interface of the compute die provides a total interconnect bandwidth of 6 TB/s across four directions. Such high interconnect bandwidth enables overlapping communication with memory access during the decoding phase or computation during the prefill phase. Each core delivers a computational power of 1.02 TFLOPS (Tensor FP16) and features 1.25 MB of SRAM.

5.1.2 Architectural Design Space Exploration. In this study, we conduct an architectural design space exploration of wafer-scale chips with the compute die described in Section 5.1.1. A single die can

Table 1: Configuration Parameters

Config. Type	Case 1	Case 2	Case 3	Case 4
#Die in X-dim	7	7	6	6
#Die in Y-dim	9	8	9	8
DRAM Bandwidth	1TB/s	1.5TB/s	2TB/s	3TB/s
DRAM Capacity	32GB	48GB	64GB	96GB
D2D Bandwidth	2.5TB/s	2.25TB/s	2TB/s	1.5TB/s

integrate varying numbers of DRAM chiplets, offering different memory capacities and bandwidth. This exploration aims to demonstrate the advantages of WSC-LLM and gain a deeper understanding of the trade-offs among computation, memory, and communication resources within wafer-scale chips. Table 1 outlines four feasible wafer-scale chip configurations within a 12-inch wafer area, which serve as the basis for our design space exploration (DSE).

5.1.3 Workloads. We choose a series of models from 7B to 175B for our experiments, including LLaMA2-7B [77], LLaMA-30B [76], LLaMA3-70B [20], and GPT-175B [13]. LLaMA, LLaMA2, LLaMA3, and GPT are the most representative and widely used decoder-only LLM model, with a similar structure to other models like Mistral [38], Falcon [11], OPT [96], BLOOM [67] and so on. LLaMA3-70B utilize group query attention (GQA), which partitions queries into groups and each group shares attention scores and a key-value (KV) matrix. In contrast, other models employ multi-head attention (MHA), in which each query maintains an independent KV matrix. We use FP16 precision for all workloads.

5.1.4 Datasets. We employ the actual production traces [1] from a cloud provider, Azure. The traces include the arrival time, input prompt size, and output token number for requests in one-hour serving. We use traces of code and conversation representing the most common scenarios in LLM inference. For the conversation (conv) dataset, the median number of prompt tokens is 1020 tokens and the average number is 1155, while the median number of decoding tokens is 129 tokens and the average number is 212. For the code dataset, the median number of prompt tokens is 1500, and the average is 2048, while the median number of decoding tokens is 13 and the average number is 28. Moreover, we increase and decrease the request load per second by adjusting the Poisson arrival rate for various model and cluster sizes.

5.1.5 Metrics. To comprehensively measure the overall performance of our system, we choose End-to-end (E2E) latency and Transaction Per Second (TPS). E2E latency represents the total response time for a user request and it includes prefill execution, decoding execution, and request waiting time. As decoding dominates LLM workloads, computation and unoverlapped communication should constitute a very low portion (<5%) of the total execution time in an efficient LLM service system. TPS represents the number of user requests processed per second.

5.2 Architectural Design Space Exploration

Fig. 10 illustrates the overall performance results of four feasible wafer-scale architectures across four LLM models and two datasets. These experiments cover variations in model architecture, input sequence length, and batch size, providing a comprehensive

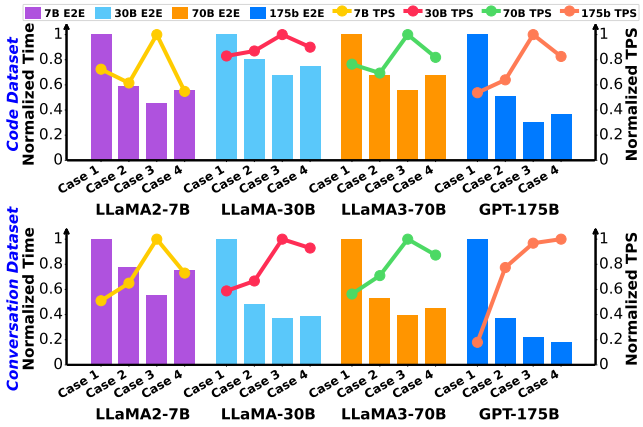


Figure 10: Overall comparisons of E2E latency and TPS among case 1, 2, 3, and 4.

and general evaluation. The wafer-scale chip with 54 dies (case 3) consistently achieves optimal performance in nearly all scenarios, demonstrating its ability to effectively adapt to variations in model architectures and workloads. This establishes case 3 as the best wafer-scale chip architecture. **The above result demonstrates that wafer-scale chips with moderate DRAM capacity per die can effectively balance computation, storage, and communication resources, delivering superior quality of service for LLM workloads.** Conversely, architectures with either excessively large or small DRAM capacity per die tend to suffer from a negative impact due to the deficiency of one or two hardware resources, leading to degraded performance.

A comparison of cases 1, 2, and 3 reveals that the overall performance of cases 1 and 2 is significantly inferior to that of case 3, particularly when evaluated on the conv dataset. This disparity can be attributed to the higher proportion of decoding phases in the conv dataset, where performance is heavily constrained by memory bandwidth and capacity. Specifically, during the decoding phase, neither enhanced computation power nor increased D2D bandwidth can compensate for insufficient DRAM bandwidth and capacity. *This finding underscores DRAM bandwidth and capacity as key factors in determining the quality of LLM service.*

When comparing case 3 and case 4, we find that although case 4 has higher DRAM bandwidth than case 3, its overall performance is inferior. This performance discrepancy arises due to its lower D2D bandwidth. During inter-die data access, the limited D2D bandwidth becomes the new bottleneck, replacing DRAM bandwidth as the primary constraint. This observation highlights a critical **insight: achieving efficient LLM service requires not only high DRAM bandwidth but also sufficient communication bandwidth. A balance between memory and communication resources is essential for optimal performance.** This principle applies not only to wafer-scale chips but also to other hardware platforms, emphasizing the importance of balanced resource allocation in achieving high-quality LLM services.

5.3 Overall Performance

We select the optimal wafer-scale chip configuration (case 3) for comparison with Splitwise [62], the SOTA LLM service system

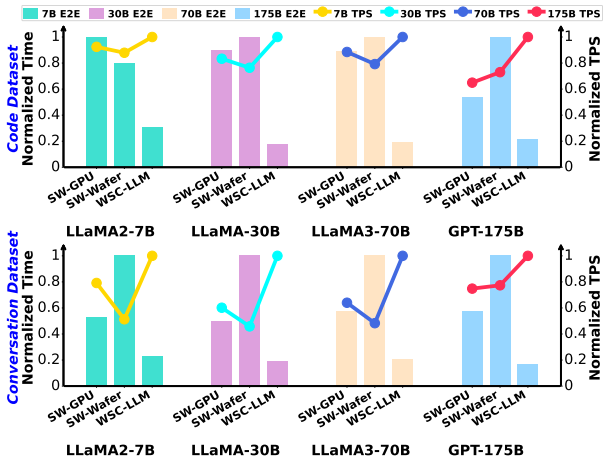


Figure 11: Overall comparisons of E2E latency and TPS between Splitwise-GPU, Splitwise-Wafer and WSC-LLM.

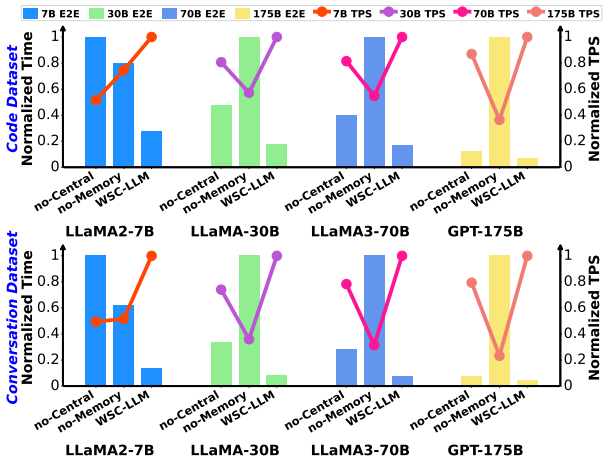


Figure 12: Overall comparisons of E2E latency and TPS among no-Central, no-Memory, and WSC-LLM.

based on GPU clusters, denoted as **SW-GPU**. To ensure a fair comparison, we use a configuration for Splitwise comprising six 8-A100-80GB GPU nodes, with inter-node bandwidth of 400 GB/s. In terms of computation resources, the total compute capability of Splitwise is 14,976 TFLOPS (Tensor FP16), which exceeds the 14,100 TFLOPS provided by the wafer-scale chip. For memory resources, Splitwise has a total DRAM capacity of 3,840 GB, also surpassing the 3,456 GB of DRAM available on the wafer-scale chip. Both systems feature a DRAM bandwidth of 2 TB/s. Additionally, we directly apply Splitwise's scheduling strategy to our wafer-scale chip (case 3) as a control experiment, denoted as **SW-Wafer**.

Fig. 11 illustrates the normalized E2E latency and TPS results on code and conversation datasets. WSC-LLM achieves **an average 3.12× improvement** in E2E latency and a 36.47% increase in TPS compared to SW-GPU, and an average 4.81× improvement in E2E latency and a 57.17% increase in TPS compared to SW-Wafer, across four LLM models and two datasets. Despite having fewer computation and memory resources, WSC-LLM demonstrates superior performance due to its highly efficient operator execution and scheduling strategies, which are specifically tailored to leverage the unique advantages of wafer-scale architecture. The performance gap between SW-GPU and SW-Wafer can be attributed both to less hardware resources (with SW-GPU having approximately 6% less compute and 11% less memory resource) and to the fact that the Splitwise strategy is designed primarily for GPU architectures, lacking the optimizations necessary for wafer-scale chips. ***These results indicate that using WSC-LLM on wafer-scale chips shows exceptional promise for LLM services.***

5.4 Ablation Studies

To evaluate the impact of different WSC-LLM's optimization strategies, we conduct experiments on a 54-die wafer-scale chip (case 3). As shown in Fig. 12, WSC-LLM is compared with two baselines, each disabling one optimization strategy. **no-Central** removes TP size search and resource allocation and placement strategy in Central Scheduler, fixing TP size at six dies for prefill and decoding instances (18 dies for GPT3-175B) and using a random placement strategy. **no-Memory** disables Memory Scheduler, requiring the generated KV cache from prefill instances to be fully transferred to decoding instances for storage.

We can observe the following phenomena: (1) Phenomenon 1: As model size increases, performance gains from the Central Scheduler gradually diminish; (2) Phenomenon 2: Conversely, benefits of the Memory Scheduler grow with the model size.

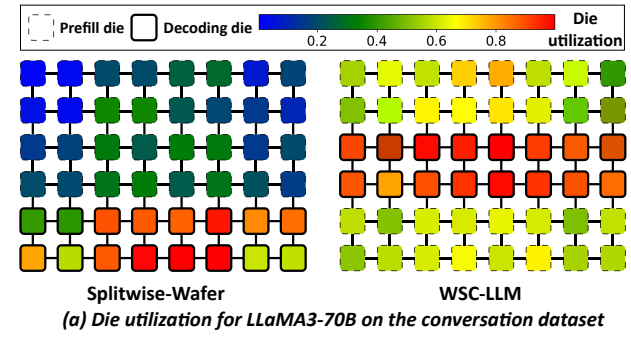
The explanation for Phenomenon 1 lies in the fact that smaller LLMs typically have smaller optimal instance size and a larger number of instances, making them deviate more from a fixed instance size and more affected by resource allocation and placement strategies. For Phenomenon 2, larger LLMs generally require more memory resource due to their increased weight parameters and KV cache. Although LLaMA3-70B reduces KV cache through GQA mechanism, its weight size is still 2.33× that of LLaMA-30B. Due to the absence of the Memory Scheduler, no-Memory fails to fully utilize DRAM resources, resulting in increased memory pressure when handling larger LLMs with higher memory demands, leading to degraded performance. ***These results confirm that by leveraging the Central Scheduler and Memory Scheduler, WSC-LLM can support high-performance LLM services across various models and workloads.***

In addition, we observe that for LLaMA-30B, LLaMA3-70B, and GPT-175B, the Memory Scheduler plays a more significant role in performance improvement whereas the Central Scheduler has a greater impact only on LLaMA2-7B. Combined with the result in Fig. 11, which show WSC-LLM achieves greater performance gains over Splitwise as model size grows, we conclude that: **The Memory Scheduler in WSC-LLM plays a more critical role than the Central Scheduler in enhancing LLM service quality.**

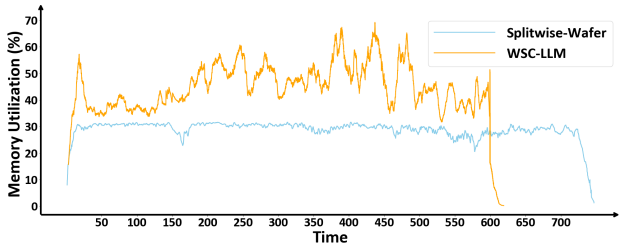
5.5 Learn from a Complete LLM Service Process

In this section, we demonstrate the advantages of WSC-LLM according to a complete real-world LLM inference process (Fig. 13) and analyze the principles of its performance improvements. **Splitwise-Wafer** serves as the baseline, representing the direct application of Splitwise's scheduling strategy to wafer-scale chips.

Fig. 13(a) presents the die utilization heatmap for LLaMA3-70B on the conversation dataset. It is evident that WSC-LLM achieves



(a) Die utilization for LLaMA3-70B on the conversation dataset



(b) DRAM utilization for LLaMA3-70B on the conversation dataset

Figure 13: Overall comparisons of die utilization and DRAM utilization for LLaMA3-70B on the conversation dataset.

significantly higher overall die utilization compared to Splitwise-Wafer, as indicated by the absence of blue or dark green dies. The performance improvement primarily stems from the designs of our Central Scheduler and Memory Scheduler:

(1) Splitwise-Wafer suffers from inefficient resource partition and placement strategies, leading to a higher total number of communication hops for edge dies. This results in frequent communication congestion, forcing computations to stall until data transmission is completed. This also explains why prefill and decoding dies in Splitwise-Wafer exhibit higher utilization in central dies while remaining underutilized at the edges. In contrast, WSC-LLM, benefiting from our Central Scheduler, achieves a more balanced utilization pattern and improved overall performance.

(2) Our Memory Scheduler can effectively leverage the idle memory resources in Splitwise-Wafer. As shown in Fig. 13(b), WSC-LLM achieves significantly higher DRAM utilization compared to Splitwise-Wafer. More DRAM space allows for storing additional KV cache of user requests, enhancing the ability to execute decoding requests in parallel and reducing the frequency of prefill dies being stalled due to insufficient DRAM space.

Furthermore, both Splitwise-Wafer and WSC-LLM exhibit higher utilization in decoding dies, highlighting the dominant role of the decoding phase. Notably, simply allocating more dies for decoding phase while reducing prefill dies does not improve LLM service quality. This is because the decoding phase is constrained by DRAM access bandwidth, and increasing dies cannot resolve it. On the contrary, it may degrade the performance of the prefill phase.

6 Discussion

6.1 Architectural Generality

As mentioned in Section 2.3, WSC-LLM is equipped with various configurable architectural parameters. For wafer-scale chips with varying parameter configurations, WSC-LLM can automatically perform end-to-end scheduling to achieve optimal LLM inference results. Notably, WSC-LLM is capable of optimizing not only for scenarios with high D2D bandwidth but also for cases with lower D2D bandwidth, as demonstrated in Section 5.1 with case 4. Moreover, the results in Section 5.3 demonstrate that WSC-LLM achieves significant performance improvements on various models including LLaMA and GPT and different model sizes from 7B to 175B. **In summary, WSC-LLM demonstrates excellent generality, adapting well to different LLM models and sizes and different architectural parameters of wafer-scale chips.**

6.2 Scalability

While wafer-scale chips offer higher integration density, their limited area constrains the computation and storage resources compared to GPU clusters with dozens or even hundreds of nodes. For LLM inference scenarios with large parameter sizes and high user request rates, utilizing multi-wafer-scale chip systems becomes imperative.

We select a wafer-scale chip array composed of 2×2 case 3 and compare its performance to Splitwise[62], which consists of twenty-four 8-A100-80GB GPU nodes, to demonstrate the scalability of WSC-LLM. For wafer-to-wafer (W2W) interconnect bandwidth, we considered two configurations: the state-of-the-art 1.8 TB/s W2W interconnect[73], denoted as WSC-LLM-18, and a configuration with 400 GB/s W2W bandwidth, matching the interconnect bandwidth of Splitwise inter-node, denoted as WSC-LLM-4. In addition, we test the LLaMA3-405B to demonstrate WSC-LLM's effective support for ultra-large models.

As shown in Fig. 14, we can observe two key phenomena. (1) Phenomenon 1: WSC-LLM consistently outperforms Splitwise in both high and low W2W bandwidth scenarios. (2) Phenomenon 2: WSC-LLM can achieve greater performance improvement on the ultra-large model LLaMA3-405B.

For Phenomenon 1, the superior performance of WSC-LLM stems from its ability to fully utilize the computational and memory resources of a single wafer to efficiently execute LLM inference. Additionally, multiple wafers can independently handle different requests in parallel, effectively minimizing the need for extensive W2W communication. In scenarios with higher W2W bandwidth, WSC-LLM can further optimize performance by employing more aggressive cross-wafer memory scheduling strategies.

For Phenomenon 2, this can be attributed to the substantial memory demands of LLaMA3-405B, which has 910GB model weights, requiring at least two GPU nodes to store a single weight replica. As a result, Splitwise incurs significant inter-node communication overhead when executing intra-instance computations, leading to performance degradation. In contrast, the memory capacity of a single wafer-scale chip (case 3) is sufficient to accommodate a complete model replica. Consequently, even WSC-LLM-4 can achieve substantial performance improvements.

7 Related Work

LLM Serving and Disaggregated Inference. In real-time online LLM service systems [10, 45, 84, 91, 100], continuous batching is a

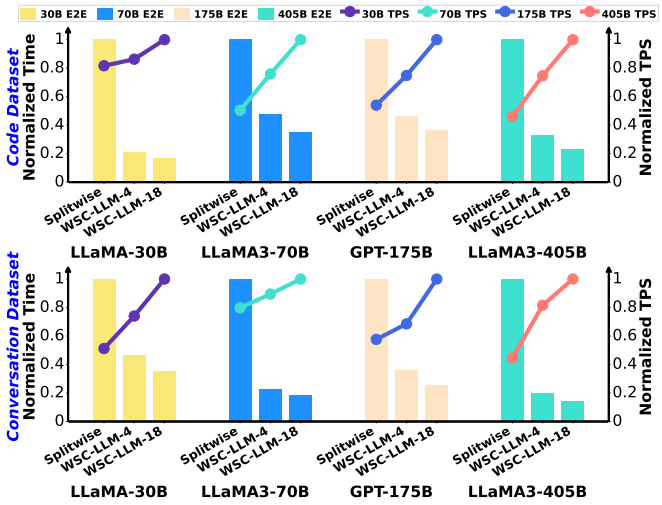


Figure 14: Overall comparisons of E2E latency and TPS among Splitwise, WSC-LLM-4, and WSC-LLM-18.

widely adopted technique that processes both the prefill and decoding phases of multiple requests within each iteration. This method aims to enhance the overall system throughput by minimizing idle hardware resources. However, since the prefill and decoding phases are constrained by computing power and memory bandwidth, respectively, continuous batching causes prefill-decoding interference. This interference leads to a trade-off within the LLM service system between optimizing the prefill and decoding phase, making it challenging to achieve simultaneous performance optimization for both phases.

Some existing LLM service systems [31, 62, 72, 100] have recognized the differences between the prefill and decoding phases and have executed them on separate devices, known as disaggregated inference. However, these efforts lack a fine-grained exploration of optimal resource allocation and do not focus on wafer-scale chips. As a result, they fail to fully leverage the flexible scheduling and high-bandwidth characteristics of wafer-scale chips, missing the opportunity to improve compute and memory utilization and to reduce additional KV cache transfer overhead.

Wafer-scale chips and Chiplet accelerators. For wafer-scale chips, existing research primarily focuses on architectural exploration [16, 22, 60, 65, 101] and layout design [39, 51, 53, 59]. However, these studies lack an understanding of the imbalances between prefill and decoding phases, and the memory management.

Regarding chiplet accelerators, research has addressed design space exploration (DSE) [15, 40, 58, 94, 98], data flow scheduling [24, 57], and SRAM optimization [25, 69]. Nevertheless, these efforts have mainly concentrated on optimizing traditional DNNs, such as CNNs, and SRAM, whereas our focus is on optimizing LLMs and DRAM utilization. Furthermore, all research overlooks the critical issue of balancing memory, computation, and communication resources within wafer-scale chip architectures.

8 Conclusion

In this work, we introduce WSC-LLM, an efficient LLM service and architecture co-exploration framework for wafer-scale chips. To the best of our knowledge, this is the first work to achieve it. Leveraging the high bandwidth and topology characteristics of wafer-scale chips, WSC-LLM introduces optimal hardware resource allocation and placement strategies, as well as KV cache memory management policies, unearthing hidden optimization opportunities. Regarding architecture, we provide a highly configurable hardware template and an accurate performance evaluator built upon existing well-established frameworks. Our experiments demonstrate that the architectures and scheduling strategies explored by WSC-LLM on wafer-scale chips significantly outperform both the SOTA LLM service system based on GPU clusters and the direct application of its scheduling strategy to wafer-scale chips. Furthermore, we provide profound insights into the design of wafer-scale architectures and the execution of LLM workloads.

Acknowledgments

This work was supported in part by the National Science and Technology Major Project under Grant 2022ZD0115200; in part by the NSFC under Grant 62125403, Grant 92464302,Grant U24B20164 and Grant 92164301; in part by Shanghai Municipal Science and Technology Major Project; in part by the Natural Science Foundation of Jiangsu Province Basic Research Program under Grant BK20243042; in part by the Beijing National Research Center for Information Science and Technology; in part by the Northern IC Technology Innovation Center (Beijing) Co., Ltd under Grant QYJS20232801B; and in part by the Beijing Advanced Innovation Center for Integrated Circuits.

References

[1] [n. d.]. Azure Public Dataset. [Online]. Available: https://github.com/Azure/AzurePublicDataset/tree/master.
[2] [n. d.]. Claude. [Online]. Available: https://claude.ai/.
[3] [n. d.]. Gemini. [Online]. Available: https://gemini.google.com/.
[4] [n. d.]. Grok. [Online]. Available: https://x.ai/blog/grok-1.5.
[5] [n. d.]. Mask / reticle. [Online]. Available: https://en.wikichip.org/wiki/mask.
[6] [n. d.]. NVIDIA GB200 NVL72: Powering the new era of computing. [Online]. Available: https://www.nvidia.com/en-us/data-center/gb200-nvl72/.
[7] [n. d.]. Nvidia H100. [Online]. Available: https://www.nvidia.com/en-us/data-center/h100.
[8] 2023. Bard, an experiment by google. [Online]. Available: https://bard.google.com/.
[9] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish, Tabarak Khan, Logan Kilpatrick, Jong Wook, Christina Kim, Yongjik Kim, Jan Hendrik, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming, Rachel Lim, Molly Lin, Stephanie

Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de, Michael Petrov, Henrique Ponde, Michael (Rai)Pokorny, Michelle Pokrass, Vitchyr H., Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B., Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[10] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).

[11] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. 2023. The Falcon Series of Open Language Models. *arXiv preprint arXiv:2311.16867* (2023).

[12] Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, Steven Hand, Dan Hurt, Michael Isard, Hyeontaek Lim, Ruoming Pang, Sudip Roy, Brennan Saeta, Parker Schuh, Ryan Sepassi, Laurent El Shafey, Chandramohan A. Thekkath, and Yonghui Wu. 2022. Pathways: Asynchronous distributed dataflow for ml. *Proceedings of Machine Learning and Systems* 4 (2022), 430–449.

[13] Tom B.Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M.Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[14] Jingwei Cai, Yuchen Wei, Zuotong Wu, Sen Peng, and Kaisheng Ma. 2023. Inter-layer scheduling space definition and exploration for tiled accelerators. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–17.

[15] Jingwei Cai, Zuotong Wu, Sen Peng, Yuchen Wei, Zhanhong Tan, Guiming Shi, Mingyu Gao, and Kaisheng Ma. 2024. Gemini: Mapping and Architecture Co-exploration for Large-scale DNN Chiplet Accelerators. In *2024 IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024*. IEEE, 156–171.

[16] Shuangliang Chen, Saptadeep Pal, and Rakesh Kumar. 2024. Waferscale network switches. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture, ISCA 2024*. IEEE, 215–229.

[17] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.

[18] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[19] Jinyi Deng, Xinru Tang, Jiahao Zhang, Yuxuan Li, Linyun Zhang, Boxiao Han, Hongjun He, Fengbin Tu, Leibo Liu, Shaojun Wei, Yang Hu, and Shouyi Yin. 2023. Towards efficient control flow handling in spatial architecture via architecting the control flow plane. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1395–1408.

[20] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Lauren Rantala-Yeary, Laurens van, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira, Robert Stojnic, Roberta Raileanu, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaoqing Ellen, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aaron Grattafiori, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco Guzmán, Frank Kanayet, Frank Seide, Gabriela Medina, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi (Jack)Zhang, Guna Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy

Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kun Huang, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Maria Tsimpoukelli, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Michael L., Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikolay Pavlovich, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Rohan Maheswari, Russ Howes, Ruty Rinott, Sai Jayesh, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish, Vishal Mangla, Vítor Albiero, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaofang Wang, Xiaojian Wu, Xiaolan Wang, Xide Xia, Xilun Wu, Xinbo Gao, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu (Sid)Wang, Yuchen Hao, Yundi Qian, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, and Zhiwei Zhao. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[21] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, Lansong Diao, Xiaoyong Liu, and Wei Lin. 2021. DAPPLE: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 431–445.

[22] Yinxiao Feng and Kaisheng Ma. 2024. Switch-Less Dragonfly on Wafers: A Scalable Interconnection Architecture based on Wafer-Scale Integration. *arXiv preprint arXiv:2407.10290* (2024).

[23] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. 2017. Tetris: Scalable and efficient neural network acceleration with 3d memory. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*. 751–764.

[24] Mingyu Gao, Xuan Yang, Jing Pu, Mark Horowitz, and Christos Kozyrakis. 2019. Tangram: Optimized coarse-grained dataflow for scalable nn accelerators. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 807–820.

[25] Raveesh Garg, Hyoukjun Kwon, Eric Qin, Yu-Hsin Chen, Tushar Krishna, and Liangzhen Lai. 2024. PipeOrgan: Efficient Inter-operation Pipelining with Flexible Spatial Organization and Interconnects. *arXiv preprint arXiv:2405.01736* (2024).

[26] Haoming Guo, Shengbin Cao, Lei Li, and Xiaofeng Zhang. 2022. A review on the mainstream through-silicon via etching methods. *Materials Science in Semiconductor Processing* 137 (2022), 106182.

[27] Kartik Hegde, Po-An Tsai, Sitao Huang, Vikas Chandra, Angshuman Parashar, and Christopher W Fletcher. 2021. Mind mappings: enabling efficient algorithm-accelerator mapping space search. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 943–958.

[28] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671* (2024).

[29] SY Hou, W Chris Chen, Clark Hu, Christine Chiu, KC Ting, TS Lin, WH Wei, WC Chiou, Vic JC Lin, Victor CY Chang, C Wang, C Wu, and D Yu. 2017. Wafer-level integration of an advanced logic-memory system through the second-generation CoWoS technology. *IEEE Transactions on Electron Devices* 64, 10 (2017), 4071–4077.

[30] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Memserve: Context caching for disaggregated llm serving with elastic memory pool. *arXiv preprint arXiv:2406.17565* (2024).

[31] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181* (2024).

[32] YuChen Hu, YuMin Liang, HsiehPin Hu, ChiaYen Tan, ChihTa Shen, ChienHsun Lee, and SY Hou. 2023. CoWoS architecture evolution for next generation HPC on 2.5 D system in package. In *2023 IEEE 73rd Electronic Components and Technology Conference, ECTC 2023*. IEEE, 1022–1026.

[33] Yang Hu, Xinhan Lin, Huizheng Wang, Zhen He, Xingmao Yu, Jiahao Zhang, Qize Yang, Zheng Xu, Sihan Guan, Jiahao Fang, Haoran Shang, Xinru Tang, Xu Dai, Shaojun Wei, and Shouyi Yin. 2024. Wafer-Scale Computing: Advancements, Challenges, and Future Perspectives. *IEEE Circuits and Systems Magazine* 24, 1 (2024), 52–81.

[34] PK Huang, CY Lu, WH Wei, Christine Chiu, KC Ting, Clark Hu, CH Tsai, SY Hou, WC Chiou, CT Wang, and Douglas Yu. 2021. Wafer level system integration of the fifth generation CoWoS®-S with high performance Si interposer at 2500 mm2. In *2021 IEEE 71st Electronic Components and Technology Conference, ECTC 2021*. IEEE, 101–104.

[35] Qijing Huang, Minwoo Kang, Grace Dinh, Thomas Norell, Aravind Kalaiah, James Demmel, John Wawrzynek, and Yakun Sophia Shao. 2021. Cosa: Scheduling by constrained optimization for spatial accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture, ISCA 2021*. IEEE, 554–566.

[36] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, and Zhifeng Chen. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).

[37] Zhihao Jia, James Thomas, Todd Warszawski, Mingyu Gao, Matei Zaharia, and Alex Aiken. 2019. Optimizing DNN computation with relaxed graph substitutions. *Proceedings of Machine Learning and Systems* 1 (2019), 27–39.

[38] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[39] Bentian Jiang, Jingsong Chen, Jinwei Liu, Lixin Liu, Fangzhou Wang, Xiaopeng Zhang, and Evangeline FY Young. 2021. CU. POKer: Placing DNNs on WSE With Optimal Kernel Sizing and Efficient Protocol Optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 6 (2021), 1888–1901.

[40] Divya Kadiyala, Saeed Rashidi, Taekyung Heo, Abhimanyu Bambhaniya, Tushar Krishna, and Alexandros Daglis. 2024. Leveraging Memory Expansion to Accelerate Large-Scale DL Training. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2024*. IEEE, 292–294.

[41] Sheng-Chun Kao, Geonhwa Jeong, and Tushar Krishna. 2020. Confuciux: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020*. IEEE, 622–636.

[42] Sheng-Chun Kao and Tushar Krishna. 2020. Gamma: Automating the hw mapping of dnn models on accelerators via genetic algorithm. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.

[43] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. 2019. Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 754–768.

[44] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. 2018. Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices* 53, 2 (2018), 461–475.

[45] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[46] John H Lau, Gary ChangFu Chen, Jones YuCheng Huang, Ricky TsunSheng Chou, Channing ChengLin Yang, HsingNing Liu, and TzyyJang Tseng. 2021. Hybrid substrate by fan-out RDL-first panel-level packaging. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 11, 8 (2021), 1301–1309.

[47] John H Lau, ChengTa Ko, KaiMing Yang, ChiaYu Peng, Tim Xia, Puru Bruce Lin, JJ Chen, Patrick PoChun Huang, HsingNing Liu, TzyyJang Tseng, Eagle Lin, and Leo Chang. 2020. Panel-level fan-out RDL-first packaging for heterogeneous integration. *IEEE Transactions on Components, Packaging and Manufacturing Technology* 10, 7 (2020), 1125–1137.

[48] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).

[49] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. 2021. Terapipe: Token-level pipeline parallelism for training large-scale language models. In *International Conference on Machine Learning*. PMLR, 6543–6552.

[50] Sean Lie. 2022. Cerebras architecture deep dive: First look inside the hw/sw co-design for deep learning: Cerebras systems. In *2022 IEEE Hot Chips 34 Symposium, HCS 2022*. IEEE Computer Society, 1–34.

[51] Jinwei Liu, Xiaopeng Zhang, Shiju Lin, Xinshi Zang, Jingsong Chen, Bentian Jiang, Martin DF Wong, and Evangeline FY Young. 2022. Partition and place finite element model on wafer-scale engine. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 631–636.

[52] Liqiang Lu, Naiqing Guan, Yuyue Wang, Liancheng Jia, Zizhang Luo, Jieming Yin, Jason Cong, and Yun Liang. 2021. Tenet: A framework for modeling tensor dataflow based on relation-centric notation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture, ISCA 2021*. IEEE, 720–733.

[53] Canhui Luo, Zhouxing Su, and Zhipeng Lü. 2023. MS-CLS: An Effective Partitioning and Placement Metaheuristic for Wafer-Scale Physics Modeling. *IEEE Transactions on Emerging Topics in Computational Intelligence* (2023).

[54] Azalia Mirhoseini, Hieu Pham, Quoc V Le, Benoit Steiner, Rasmus Larsen, Yuefeng Zhou, Naveen Kumar, Mohammad Norouzi, Samy Bengio, and Jeff Dean. 2017. Device placement optimization with reinforcement learning. In *International conference on machine learning*. PMLR, 2430–2439.

[55] Gordon E Moore. 1998. Cramming more components onto integrated circuits. *Proc. IEEE* 86, 1 (1998), 82–85.

[56] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.

[57] Marcelo Orenes-Vera, Esin Tureci, Margaret Martonosi, and David Wentzlaff. 2023. DCRA: A distributed chiplet-based reconfigurable architecture for irregular applications. *arXiv preprint arXiv:2311.15443* (2023).

[58] Marcelo OrenesVera, Esin Tureci, Margaret Martonosi, and David Wentzlaff. 2024. Muchisim: A simulation framework for design exploration of multi-chip manycore systems. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2024*. IEEE, 48–60.

[59] Sarp Özdemir, Mohammad Khasawneh, Smriti Rao, and Patrick H Madden. 2022. Kernel mapping techniques for deep learning neural network accelerators. In *Proceedings of the 2022 International Symposium on Physical Design*. 21–28.

[60] Saptadeep Pal, Jingyang Liu, Irina Alam, Nicholas Cebry, Haris Suhail, Shi Bu, Subramanian S Iyer, Sudhakar Pamarti, Rakesh Kumar, and Puneet Gupta. 2021. Designing a 2048-chiplet, 14336-core waferscale processor. In *2021 58th ACM/IEEE Design Automation Conference, DAC 2021*. IEEE, 1183–1188.

[61] Saptadeep Pal, Daniel Petrisko, Matthew Tomei, Puneet Gupta, Subramanian S Iyer, and Rakesh Kumar. 2019. Architecting waferscale processors-a GPU case study. In *2019 IEEE International Symposium on High Performance Computer Architecture, HPCA 2019*. IEEE, 250–263.

[62] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2023. Splitwise: Efficient generative llm inference using phase splitting. *Power* 400, 700W (2023), 1–75.

[63] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023), 606–624.

[64] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[65] Saeed Rashidi, William Won, Sudarshan Srinivasan, Puneet Gupta, and Tushar Krishna. 2024. FRED: Flexible REduction-Distribution Interconnect and Communication Implementation for Wafer-Scale Distributed Training of DNN Models. *arXiv preprint arXiv:2406.19580* (2024).

[66] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[67] Teven Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander Rush, Stella Biderman, Albert Webson, Pawan Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Aji, Amit Alfassy, Anna Rogers, Ariel Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel Strien, David Adelani, Dragomir Radev, Eduardo Ponferrada, Efrat Levkovizh, Ethan Kim, Eyal Natan, Francesco Toni, Gérard Dupont, Germán Kruszewski, Giada Pistilli, Hady Elsahar, Hamza Benyamina, Hieu Tran, Ian Yu, Idris Abdulmumin, Isaac Johnson, Itziar Gonzalez-Dios, Javier Rosa, Jenny Chim, Jesse Dodge, Jian Zhu, Jonathan Chang, Jörg Frohberg, Joseph Tobing, Joydeep Bhattacharjee, Khalid Almubarak, Kimbo Chen, Kyle Lo, Leandro Werra, Leon Weber, Long Phan, Loubna allal, Ludovic Tanguy, Manan Dey, Manuel Muñoz, Maraim Masoud, María Grandury, Mario Šaško, Max Huang, Maximin Coavoux, Mayank Singh, Mike Jiang, Minh Vu, Mohammad Jauhar, Mustafa Ghaleb, Nishant Subramani, Nora Kassner, Nurulaqilla Khamis, Olivier Nguyen, Omar Espejel, Ona Gibert, Paulo, Peter Henderson, Pierre Colombo, Priscilla Amuok, Quentin Lhoest, Rheza Harliman, Rishi Bommasani, Roberto López, Rui Ribeiro, Salomey Osei, Sampo Pyysalo, Sebastian Nagel, Shamik Bose, Shamsuddeen Muhammad, Shanya Sharma, Shayne Longpre, Somaieh Nikpoor, Stanislav Silberberg, Suhas Pai, Sydney Zink, Tiago Torrent, Timo Schick, Tristan Thrush, Valentin Danchev, Vassilina Nikoulina, Veronika Laippala, Violette Lepercq, Vrinda Prabhu, Zaid Alyafeai, Zeerak Talat, Arun Raja, Benjamin Heinzerling, Chenglei Si, Davut Taşar, Elizabeth Salesky, Sabrina Mielke, Wilson Lee, Abheesht Sharma, Andrea Santilli, Antoine Chaffin, Arnaud Stiegler, Debajyoti Datta, Eliza Szczechla, Gunjan Chhablani, Han Wang, Harshit Pandey, Hendrik Strobelt, Jason Fries, Jos Rozen, Leo Gao, Lintang Sutawika, M Bari, Maged Al-shaibani, Matteo Manica, Nihal Nayak, Ryan Teehan, Samuel Albanie, Sheng Shen, Srulik Ben-David, Stephen Bach, Taewoon Kim, Tali Bers, Thibault Fevry, Trishala Neeraj, Urmish Thakker, Vikas Raunak, Xiangru Tang, Zheng-Xin Yong, Zhiqing Sun, Shaked Brody, Yallow Uri, Hadar Tojarieh, Adam Roberts, Hyung Chung, Jaesung Tae, Jason Phang, Ofir Press, Conglong Li, Deepak Narayanan, Hatim Bourfoune, Jared Casper, Jeff Rasley, Max Ryabinin, Mayank Mishra, Minjia Zhang, Mohammad Shoeybi, Myriam Peyrounette, Nicolas Patry, Nouamane Tazi, Omar Sanseviero, Patrick Platen, Pierre Cornette, Pierre Lavallée, Rémi Lacroix, Samyam Rajbhandari, Sanchit Gandhi, Shaden Smith, Stéphane Requena, Suraj Patil, Tim Dettmers, Ahmed Baruwa, Amanpreet Singh, Anastasia Cheveleva, Anne-Laure Ligozat, Arjun Subramonian, Aurélie Névéol, Charles Lovering, Dan Garrette, Deepak Tunuguntla, Ehud Reiter, Ekaterina Taktasheva, Ekaterina Voloshina, Eli Bogdanov, Genta Winata, Hailey Schoelkopf, Jan-Christoph Kalo, Jekaterina Novikova, Jessica Forde, Jordan Clive, Jungo Kasai, Ken Kawamura, Liam Hazan, Marine Carpuat, Miruna Clinciu, Najoung Kim, Newton Cheng, Oleg Serikov, Omer Antverg, Oskar Wal, Rui Zhang, Ruochen Zhang, Sebastian Gehrmann, Shachar Mirkin, Shani Pais, Tatiana Shavrina, Thomas Scialom, Tian Yun, Tomasz Limisiewicz, Verena Rieser, Vitaly Protasov, Vladislav Mikhailov, Yada Pruksachatkun, Yonatan Belinkov, Zachary Bamberger, Zdeněk Kasner, Alice Rueda, Amanda Pestana, Amir Feizpour, Ammar Khan, Amy Faranak, Ana Santos, Anthony Hevia, Antigona Unldreaj, Arash Aghagol, Arezoo Abdollahi, Aycha Tammour, Azadeh HajiHosseini, Bahareh Behroozi, Benjamin Ajibade, Bharat Saxena, Carlos Ferrandis, Daniel McDuff, Danish Contractor, David Lansky, Davis David, Douwe Kiela, Duong Nguyen, Edward Tan, Emi Baylor, Ezinwanne Ozoani, Fatima Mirza, Frankline Ononiwu, Habib Rezanejad, Hessie Jones, Indrani Bhattacharya, Irene Solaiman, Irina Sedenko, Isar Nejadgholi, Jesse Passmore, Josh Seltzer, Julio Sanz, Livia Dutra, Mairon Samagaio, Maraim Elbadri, Margot Mieskes, Marissa Gerchick, Martha Akinlolu, Michael McKenna, Mike Qiu, Muhammed Ghauri, Mykola Burynok, Nafis Abrar, Nazneen Rajani, Nour Elkott, Nour Fahmy, Olanrewaju Samuel, Ran An, Rasmus Kromann, Ryan Hao, Samira Alizadeh, Sarmad Shubber, Silas Wang, Sourav Roy, Sylvain Viguier, Thanh Le, Tobi Oyebade, Trieu Le, Yoyo Yang, Zach Nguyen, Abhinav Kashyap, Alfredo Palasciano, Alison Callahan, Anima Shukla, Antonio Miranda-Escalada, Ayush Singh, Benjamin Beilharz, Bo Wang, Caio Brito, Chenxi Zhou, Chirag Jain, Chuxin Xu, Clémentine Fourrier, Daniel Periñán, Daniel Molano, Dian Yu, Enrique Manjavacas, Fabio Barth, Florian Fuhrimann, Gabriel Altay, Giyaseddin Bayrak, Gully Burns, Helena Vrabec, Imane Bello, Ishani Dash, Jihyun Kang, John Giorgi, Jonas Golde, Jose Posada, Karthik Sivaraman, Lokesh Bulchandani, Lu Liu, Luisa Shinzato, Madeleine Bykhovetz, Maiko Takeuchi, Marc Pàmies, Maria Castillo, Marianna Nezhurina, Mario Sänger, Matthias Samwald, Michael Cullan, Michael Weinberg, Michiel Wolf, Mina Mihaljcic, Minna Liu, Moritz Freidank, Myungsun Kang, Natasha Seelam, Nathan Dahlberg, Nicholas Broad, Nikolaus Muellner, Pascale Fung, Patrick Haller, Ramya Chandrasekhar, Renata Eisenberg, Robert Martin, Rodrigo Canalli, Rosaline Su, Ruisi Su, Samuel Cahyawijaya, Samuele Garda, Shlok Deshmukh, Shubhanshu Mishra, Sid Kiblawi, Simon Ott, Sinee Sang-aroonsiri, Srishti Kumar, Stefan Schweter, Sushil Bharati, Tanmay Laud, Théo Gigant, Tomoya Kainuma, Wojciech Kusa, Yanis Labrak, Yash Bajaj, Yash Venkatraman, Yifan Xu, Yingxin Xu, Yu Xu, Zhe Tan, Zhongli Xie, Zifan Ye, Mathilde Bras, Younes Belkada, and Thomas Wolf. 2023. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100v4* (2023).

[68] Johannes Schemmel, Daniel Brüderle, Andreas Grübl, Matthias Hock, Karlheinz Meier, and Sebastian Millner. 2010. A wafer-scale neuromorphic hardware system for large-scale neural modeling. In *2010 IEEE International Symposium on Circuits and Systems , ISCAS 2010*. IEEE, 1947–1950.

[69] Yakun Sophia Shao, Jason Clemons, Rangharajan Venkatesan, Brian Zimmer, Matthew Fojtik, Nan Jiang, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney,

Priyanka Raina, S G.Tell, Y Zhang, J.Dally, Joel Emer, C Gray, B Khailany, and S W.Keckler. 2019. Simba: Scaling deep-learning inference with multi-chip-module-based architecture. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 14–27.

[70] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, R Sepassi, and B Hechtman. 2018. Mesh-tensorflow: Deep learning for supercomputers. *Advances in neural information processing systems* 31 (2018).

[71] Mingcong Song, Xinru Tang, Fengfan Hou, Jing Li, Wei Wei, Yipeng Ma, Runqiu Xiao, Hongjie Si, Dingcheng Jiang, Shouyi Yin, Yang Hu, and Guoping Long. 2024. Tackling the dynamicity in a production llm serving system with sota optimizations via hybrid prefill/decode/verify scheduling on efficient meta-kernels. *arXiv preprint arXiv:2412.18106* (2024).

[72] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. *arXiv preprint arXiv:2403.01876* (2024).

[73] Emil Talpes, Douglas Williams, and Debjit Das Sarma. 2022. Dojo: The microarchitecture of tesla's exa-scale computer. In *2022 IEEE Hot Chips 34 Symposium, HCS 2022*. IEEE Computer Society, 1–28.

[74] Zhanhong Tan, Hongyu Cai, Runpei Dong, and Kaisheng Ma. 2021. Nn-baton: Dnn workload orchestration and chiplet granularity exploration for multichip accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture, ISCA 2021*. IEEE, 1013–1026.

[75] Romal Thoppilan, Daniel Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, HengTze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, YaGuang Li, Hongrae Lee, Huaixiu Steven, Amin Ghafouri, Marcelo Menegali, Yanping Huang, Maxim Krikun, Dmitry Lepikhin, James Qin, Dehao Chen, Yuanzhong Xu, Zhifeng Chen, Adam Roberts, Maarten Bosma, Vincent Zhao, Yanqi Zhou, ChungChing Chang, Igor Krivokon, Will Rusch, Marc Pickett, Pranesh Srinivasan, Laichee Man, Kathleen MeierHellstern, Meredith Ringel, Tulsee Doshi, Renelito Delos, Toju Duke, Johnny Soraker, Ben Zevenbergen, Vinodkumar Prabhakaran, Mark Diaz, Ben Hutchinson, Kristen Olson, Alejandra Molina, Erin HoffmanJohn, Josh Lee, Lora Aroyo, Ravi Rajakumar, Alena Butryna, Matthew Lamm, Viktoriya Kuzmina, Joe Fenton, Aaron Cohen, Rachel Bernstein, Ray Kurzweil, Blaise AgueraArcas, Claire Cui, Marian Croak, Ed Chi, and Quoc Le. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239* (2022).

[76] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, MarieAnne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, A Rodriguez, A Joulin, E Grave, and G Lample. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[77] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Koura, MarieAnne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Smith, Ranjan Subramanian, Xiaoqing Ellen, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[78] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[79] Swagath Venkataramani, Ashish Ranjan, Subarno Banerjee, Dipankar Das, Sasikanth Avancha, Ashok Jagannathan, Ajaya Durg, Dheemanth Nagaraj, Bharat Kaul, Pradeep Dubey, and A Raghunathan. 2017. Scaledeep: A scalable compute architecture for learning and evaluating deep networks. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 13–26.

[80] Rangharajan Venkatesan, Yakun Sophia Shao, Miaorong Wang, Jason Clemons, Steve Dai, Matthew Fojtik, Ben Keller, Alicia Klinefelter, Nathaniel Pinckney, Priyanka Raina, Yanqing Zhang, B Zimmer, W J. Dally, J Emer, Stephen W. Keckler, and Brucek Khailany. 2019. Magnet: A modular accelerator generator for neural networks. In *2019 IEEE/ACM International Conference on Computer-Aided Design, ICCAD 2019*. IEEE, 1–8.

[81] Huizheng Wang, Qize Yang, Taiquan Wei, Xingmao Yu, Chengran Li, Jiahao Fang, Guangyang Lu, Xu Dai, Liang Liu, Shenfei Jiang, Yang Hu, Shouyi Yin, and Shaojun Wei. 2024. TMAC: Training-targeted Mapping and Architecture Co-exploration for Wafer-scale Chips. *Integrated Circuits and Systems* (2024).

[82] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. 2024. TACOS: Topology-Aware Collective Algorithm Synthesizer for Distributed Machine Learning. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 856–870.

[83] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2023*. IEEE, 283–294.

[84] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-context Large Language Models with Elastic Sequence Parallelism. *arXiv preprint arXiv:2404.09526* (2024).

[85] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920* (2023).

[86] Qingcheng Xiao, Size Zheng, Bingzhe Wu, Pengcheng Xu, Xuehai Qian, and Yun Liang. 2021. Hasco: Towards agile hardware and software co-design for tensor computation. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture, ISCA 2021*. IEEE, 1055–1068.

[87] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Hongjun Choi, Blake Hechtman, and Shibo Wang. 2020. Automatic cross-replica sharding of weight update in data-parallel training. *arXiv preprint arXiv:2004.13336* (2020).

[88] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. Gspmd: general and scalable parallelization for ml computation graphs. *arXiv preprint arXiv:2105.04663* (2021).

[89] Zheng Xu, Xu Dai, Shaojun Wei, Shouyi Yin, and Yang Hu. 2024. GSPO: A Graph Substitution and Parallelization Joint Optimization Framework for DNN Inference. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

[90] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, Ankita Nayak, Steven Bell, Kaidi Cao, Heonjae Ha, Priyanka Raina, C Kozyrakis, and M Horowitz. 2020. Interstellar: Using halide's scheduling language to analyze dnn accelerators. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 369–383.

[91] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 22*. 521–538.

[92] Jinhui Yuan, Xinqi Li, Cheng Cheng, Juncheng Liu, Ran Guo, Shenghang Cai, Chi Yao, Fei Yang, Xiaodong Yi, Chuan Wu, H Zhang, and Zhao J. 2021. Oneflow: Redesign the distributed deep learning framework from scratch. *arXiv preprint arXiv:2110.15032* (2021).

[93] Hao Zhang, Yuan Li, Zhijie Deng, Xiaodan Liang, Lawrence Carin, and Eric Xing. 2020. Autosync: Learning to synchronize for data-parallel distributed deep learning. *Advances in Neural Information Processing Systems* 33 (2020), 906–917.

[94] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. Llmcompass: Enabling efficient hardware design for large language model inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture, ISCA 2024*. IEEE, 1080–1096.

[95] Min Zhang, Fei Qin, Si Chen, Yanwei Dai, Pei Chen, and Tong An. 2022. Protrusion of through-silicon-via (TSV) copper with double annealing processes. *Journal of Electronic Materials* 51, 5 (2022), 2433–2449.

[96] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).

[97] Zhen Zhang, Shuai Zheng, Yida Wang, Justin Chiu, George Karypis, Trishul Chilimbi, Mu Li, and Xin Jin. 2022. MiCS: near-linear scaling for training gigantic model on public cloud. *arXiv preprint arXiv:2205.00119* (2022).

[98] Xiaoyan Zhao, Jiale Zhang, Junna Zhang, Peiyan Yuan, Hu Jin, and Xiangyang Li. 2023. CooCo: A Collaborative Offloading and Resource Configuration Algorithm in Edge Networks. *IEEE Internet of Things Journal* (2023).

[99] Shixuan Zheng, Xianjue Zhang, Leibo Liu, Shaojun Wei, and Shouyi Yin. 2022. Atomic dataflow based graph-level workload orchestration for scalable DNN accelerators. In *2022 IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022*. IEEE, 475–489.

[100] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670* (2024).

[101] Jingchen Zhu, Chenhao Xue, Yiqi Chen, Zhao Wang, and Guangyu Sun. 2024. Theseus: Towards High-Efficiency Wafer-Scale Chip Design Space Exploration for Large Language Models. *arXiv preprint arXiv:2407.02079* (2024).